

icfes: competencia: - formulacion_ejecucion nivel_dificultad: 3 contenido: categoria: geometria tipo: no_generico contexto: matematico eje_axial: eje2 componente: geometrico_metrico

```
options(OutDec = ".") # Asegurar punto decimal en este chunk

# Establecer semilla aleatoria
set.seed(sample(1:10000, 1))

# Aleatorizamos los nombres para contextualizar el problema
nombres <- c("Camilo", "Andrés", "Sofía", "Manuel", "Laura", "Carlos",
             "Daniela", "Miguel", "Valentina", "Eduardo", "Natalia",
             "José", "Isabella", "Gabriel", "Mariana", "Santiago", "Lucía",
             "Alejandro", "Catalina", "Mateo", "Valeria", "Sebastián", "Juliana")
nombre <- sample(nombres, 1)

# Aleatorizamos los tipos de recipientes
recipientes <- c("cilindro", "tubo", "conducto", "tanque cilíndrico",
                "recipiente cilíndrico", "contenedor cilíndrico",
                "ducto cilíndrico", "depósito cilíndrico", "cañería cilíndrica")
recipiente <- sample(recipientes, 1)

# Aleatorizar adjetivos para el cilindro interno
adjetivos_interno <- c("interno", "hueco", "vacío", "interior", "central", "medio")
adjetivo_interno <- sample(adjetivos_interno, 1)

# Aleatorizar líquidos
liquidos <- c("aceite", "combustible", "líquido", "fluido", "agua", "refrigerante",
             "solución", "mezcla", "sustancia")
liquido <- sample(liquidos, 1)

# Aleatorizar lo que se desea calcular
calculos <- c("la cantidad de", "el volumen de", "cuánto", "qué cantidad de",
             "cuántos litros de", "qué volumen de", "la capacidad de")
calculo <- sample(calculos, 1)

# Aleatorizar verbos para la acción
verbos <- c("llenar", "contener", "almacenar", "ocupar", "requerir")
verbo <- sample(verbos, 1)

# Aleatorizar medidas del cilindro (manteniendo consistencia matemática)
# Primero generamos el radio interno (entre 0.1 y 0.4 metros)
r_int <- round(runif(1, 0.1, 0.4), 2)

# Luego generamos el radio externo, asegurando que sea mayor que el interno
# Radio externo entre 1.5 y 2.5 veces el radio interno
factor_radio <- runif(1, 1.5, 2.5)
r_ext <- round(r_int * factor_radio, 2)

# Calculamos el diámetro externo (exactamente 2 veces el radio externo)
d_ext <- 2 * r_ext

# Altura del cilindro (entre 0.5 y 4 metros)
altura <- round(runif(1, 0.5, 4), 2)
```

```

# Calcular el grosor de la pared del cilindro
grosor <- r_ext - r_int

# Verificar coherencia física
if (grosor < 0.05) {
  # Asegurar un grosor mínimo de 0.05 unidades
  grosor <- round(runif(1, 0.05, 0.2), 2)
  r_ext <- r_int + grosor
  # Asegurar que el diámetro externo sea exactamente 2 veces el radio externo
  d_ext <- 2 * r_ext
}

# Calcular el volumen necesario (es el volumen del cilindro interior)
volumen <- round(pi * (r_int^2) * altura, 2)

# Aleatorizar unidades de medida
unidades_longitud <- c("m", "cm", "dm")
unidad <- sample(unidades_longitud, 1)

# Ajustar valores según la unidad
factor_conversion <- 1
if (unidad == "cm") {
  factor_conversion <- 100
  d_ext <- d_ext * factor_conversion
  r_int <- r_int * factor_conversion
  altura <- altura * factor_conversion
  volumen <- volumen * (factor_conversion^3)
  r_ext <- r_ext * factor_conversion
} else if (unidad == "dm") {
  factor_conversion <- 10
  d_ext <- d_ext * factor_conversion
  r_int <- r_int * factor_conversion
  altura <- altura * factor_conversion
  volumen <- volumen * (factor_conversion^3)
  r_ext <- r_ext * factor_conversion
}

# Redondear todos los valores para mayor claridad
d_ext <- round(d_ext, 1)
r_int <- round(r_int, 1)
altura <- round(altura, 1)
r_ext <- round(r_ext, 1)
volumen <- round(volumen, 1)

# La respuesta correcta es "C) Altura del cilindro"
# Ya que para calcular el volumen necesitaríamos conocer la altura
opcion_correcta <- 3 # Índice de "Altura del cilindro" en el vector de opciones

# Vector de solución para r-exams (1 para la correcta, 0 para las incorrectas)
solucion <- c(0, 0, 1, 0)

```

```

options(OutDec = ".") # Asegurar punto decimal en este chunk

```

```

# Código Python para generar el cilindro

```

```

codigo_python <- paste0("
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.patches import Ellipse, Rectangle, Arrow, FancyArrowPatch

# Parámetros del cilindro
radio_interno = ", r_int, " # ", unidad, "
grosor = ", r_ext - r_int, " # ", unidad, "
radio_externo = radio_interno + grosor # ", r_ext, " ", unidad, "
altura = ", altura, " # ", unidad, "
diametro_externo = 2 * radio_externo # ", d_ext, " ", unidad, "
unidad = '"', unidad, '"

# Calcular la relación altura/radio para ajustar dinámicamente la visualización
relacion_altura_radio = altura / radio_externo

# Ajustar el tamaño de la figura según la relación altura/radio
# Si el cilindro es alto y delgado, aumentamos la altura de la figura
if relacion_altura_radio > 5:
    fig_height = 8 # Mayor altura para cilindros muy altos
    fig_width = 6
elif relacion_altura_radio > 3:
    fig_height = 7
    fig_width = 6
else:
    fig_height = 6
    fig_width = 5.5

# Crear figura y ejes con tamaño adaptativo
fig, ax = plt.subplots(figsize=(fig_width, fig_height))

# Calcular márgenes dinámicos basados en las proporciones del cilindro
margen_x = max(0.5, min(1.5, 5/relacion_altura_radio)) * radio_externo
margen_y_inferior = max(0.15, min(0.3, 1/relacion_altura_radio)) * altura
margen_y_superior = max(0.2, min(0.5, 2/relacion_altura_radio)) * altura

# Ajustar límites del gráfico dinámicamente
ax.set_xlim(-radio_externo - margen_x, radio_externo + margen_x)
ax.set_ylim(-margen_y_inferior, altura + margen_y_superior)

# Desactivar la relación de aspecto igual para cilindros muy altos
if relacion_altura_radio > 4:
    # No usar axis('equal') para cilindros muy altos
    pass
else:
    ax.axis('equal') # Mantener para cilindros con proporciones normales

ax.axis('off') # Ocultar ejes

# Factor de compresión para la perspectiva
factor_compresion = 0.4

# Dibujar el cilindro externo

```

```

# Base inferior
ax.add_patch(Ellipse((0, 0), width=2*radio_externo, height=2*radio_externo*factor_compresion,
                    fill=False, edgecolor='black', linestyle='-'))

# Base superior
ax.add_patch(Ellipse((0, altura), width=2*radio_externo, height=2*radio_externo*factor_compresion,
                    fill=False, edgecolor='black', linestyle='-'))

# Laterales externos
ax.plot([-radio_externo, -radio_externo], [0, altura], 'k-')
ax.plot([radio_externo, radio_externo], [0, altura], 'k-')

# Dibujar el cilindro interno
# Base inferior interna (línea punteada)
ax.add_patch(Ellipse((0, 0), width=2*radio_interno, height=2*radio_interno*factor_compresion,
                    fill=False, edgecolor='black', linestyle='--'))

# Base superior interna (línea punteada)
ax.add_patch(Ellipse((0, altura), width=2*radio_interno, height=2*radio_interno*factor_compresion,
                    fill=False, edgecolor='black', linestyle='--'))

# Laterales internos (líneas punteadas)
ax.plot([-radio_interno, -radio_interno], [0, altura], 'k--')
ax.plot([radio_interno, radio_interno], [0, altura], 'k--')

# Calcular espaciado adaptativo para etiquetas
espacio_etiqueta_superior = min(0.15 * altura, max(5, 0.05 * altura))
espacio_etiqueta_inferior = min(0.15 * altura, max(5, 0.05 * altura))

# Diámetro externo en la parte superior - Ajustado para mejor visibilidad
# Primero dibujamos la flecha
ax.annotate('',
            xy=(-radio_externo, altura + espacio_etiqueta_superior/2),
            xytext=(radio_externo, altura + espacio_etiqueta_superior/2),
            arrowprops=dict(arrowstyle='<->', color='black', lw=1.5))

# Luego colocamos la etiqueta encima de la flecha con fondo blanco
ax.annotate(f'Diámetro externo = {diametro_externo} {unidad}',
            xy=(0, altura + espacio_etiqueta_superior),
            xytext=(0, altura + espacio_etiqueta_superior),
            ha='center', va='center', fontsize=10,
            bbox=dict(boxstyle='round,pad=0.3', fc='white', ec='none', alpha=0.8))

# Calcular posición adaptativa para etiqueta de radio interno
# Si el cilindro es muy alto, colocar la etiqueta más cerca del borde superior
if relacion_altura_radio > 4:
    pos_etiqueta_radio_interno = altura - altura * 0.05
else:
    pos_etiqueta_radio_interno = altura - min(0.1 * altura, radio_externo)

# Radio interno - flecha precisa con etiqueta adaptativa
ax.annotate('',
            xy=(radio_interno, altura),

```

```

        xytext=(0, altura),
        arrowprops=dict(arrowstyle='->', color='black', lw=1))

# Etiqueta de radio interno con fondo blanco para mayor visibilidad
ax.annotate(f'Radio interno = {radio_interno} {unidad}',
            xy=(radio_interno/2, pos_etiqueta_radio_interno),
            xytext=(radio_interno/2, pos_etiqueta_radio_interno),
            ha='center', va='center', fontsize=9,
            bbox=dict(boxstyle='round,pad=0.2', fc='white', ec='none', alpha=0.8))

# Calcular posición adaptativa para la etiqueta de grosor
# Para cilindros muy altos, usar una posición relativa
pos_grosor_y = altura * 0.5 # Siempre en la mitad de la altura

# Grosor - Reposicionado para mejor visibilidad
ax.annotate(f'Grosor = {grosor} {unidad}',
            xy=(-(radio_interno + grosor/2), pos_grosor_y),
            xytext=(-radio_externo - margen_x * 0.5, pos_grosor_y),
            ha='right', va='center', fontsize=9,
            bbox=dict(boxstyle='round,pad=0.2', fc='white', ec='none', alpha=0.8),
            arrowprops=dict(arrowstyle='->', color='black', lw=1))

# Línea horizontal para mostrar el grosor - engrosada para mayor visibilidad
ax.plot([-radio_externo, -radio_interno], [pos_grosor_y, pos_grosor_y], 'k-', lw=2.5)
# Marcas verticales más visibles
ax.plot([-radio_externo, -radio_externo], [pos_grosor_y-0.02*altura, pos_grosor_y+0.02*altura], 'k-', lw=1)
ax.plot([-radio_interno, -radio_interno], [pos_grosor_y-0.02*altura, pos_grosor_y+0.02*altura], 'k-', lw=1)

# Añadir sombreado para destacar el grosor
import matplotlib.patches as patches
rect = patches.Rectangle(
    (-radio_externo, pos_grosor_y-0.02*altura),
    grosor,
    0.04*altura,
    linewidth=0,
    color='lightgray',
    alpha=0.5
)
ax.add_patch(rect)

# Calcular posición adaptativa para la etiqueta de altura
# Para cilindros muy altos, ajustar la posición horizontal
pos_altura_x = radio_externo + margen_x * 0.5

# Altura - etiqueta y flecha adaptativas
ax.text(pos_altura_x, altura/2, 'Altura',
        ha='center', va='center', fontsize=10, fontweight='bold',
        bbox=dict(boxstyle='round,pad=0.2', fc='white', ec='none', alpha=0.8))

# Flecha bidireccional para la altura
ax.annotate('',
            xy=(radio_externo + margen_x * 0.25, altura),
            xytext=(radio_externo + margen_x * 0.25, 0),

```

```

        arrowprops=dict(arrowstyle='<->', color='black', lw=1.5))

# Líneas horizontales para marcar los límites de la altura
ax.plot([radio_externo, radio_externo + margen_x * 0.4], [0, 0], 'k-', lw=1)
ax.plot([radio_externo, radio_externo + margen_x * 0.4], [altura, altura], 'k-', lw=1)

# Radio externo - flecha precisa con fondo blanco
ax.annotate('',
            xy=(radio_externo, 0),
            xytext=(0, 0),
            arrowprops=dict(arrowstyle='->', color='black', lw=1.5))

# Etiqueta de radio externo con fondo blanco
ax.annotate(f'Radio externo = {radio_externo} {unidad}',
            xy=(radio_interno, -espacio_etiqueta_inferior/2),
            xytext=(radio_interno, -espacio_etiqueta_inferior/2),
            ha='center', va='center', fontsize=10,
            bbox=dict(boxstyle='round,pad=0.3', fc='white', ec='none', alpha=0.8))

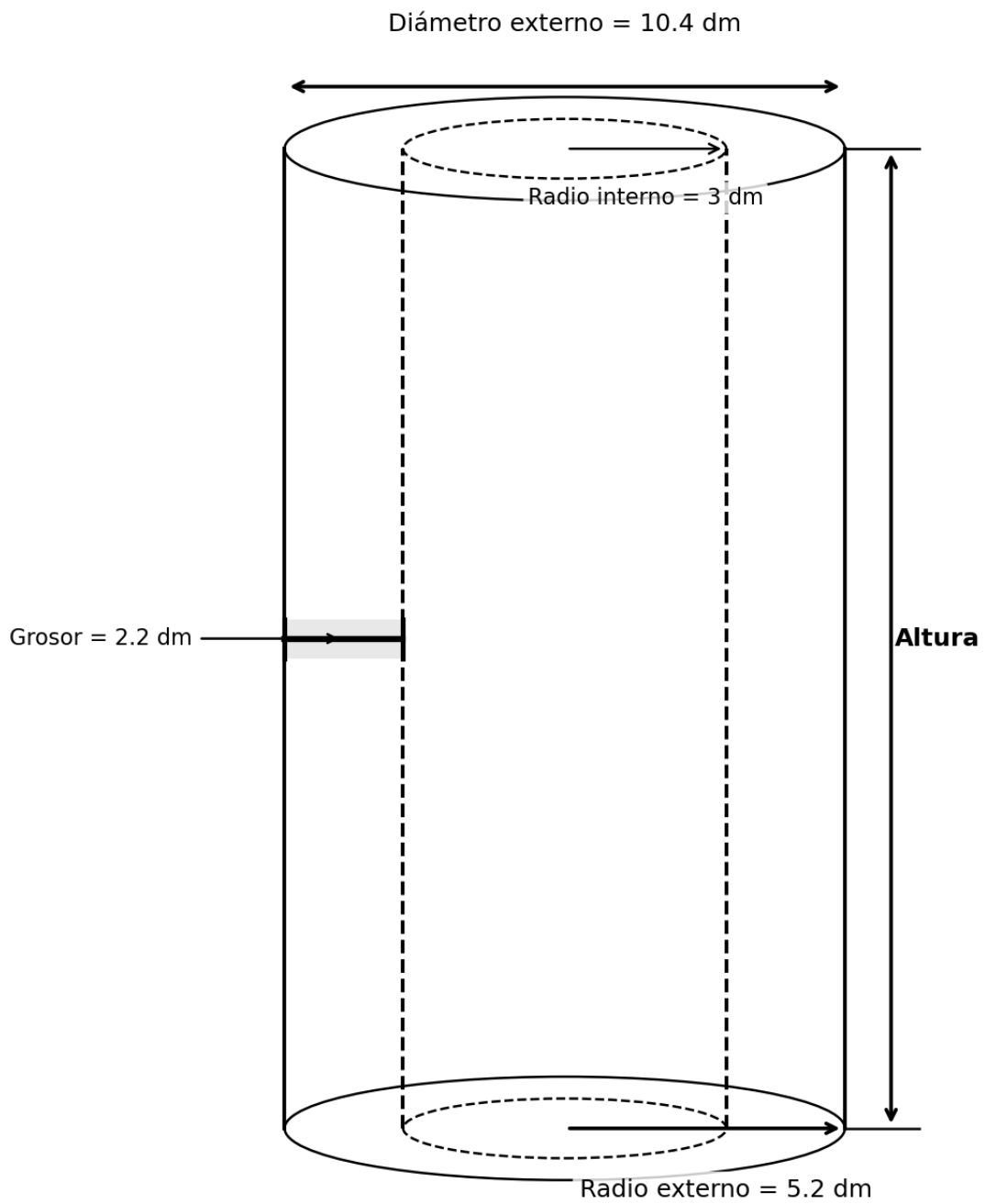
# Optimizar el layout para eliminar espacios vacíos
plt.tight_layout(pad=0.5)
# Guardar con recorte ajustado y mayor resolución para mantener calidad
plt.savefig('cilindro_hueco.png', dpi=180, bbox_inches='tight', pad_inches=0.1)
plt.close()
")

# Ejecutar código Python para generar la figura
py_run_string(codigo_python)

```

Question

Mateo desea saber cuánto fluido se necesita para llenar el depósito cilíndrico interno, pero solamente cuenta con las medidas de las dimensiones que muestra la figura.



Dimensión	Valor	Unidad
Radio interno	3	dm
Radio externo	5,2	dm
Diámetro externo	10,4	dm
Grosor	2,2	dm

¿Cuál medida le falta a Mateo para hallar la cantidad deseada?

Answerlist

- Radio externo.
- Diámetro externo.
- Altura del cilindro.
- Perímetro del cilindro.

Solution

La respuesta correcta es: **Altura del cilindro.**

Para calcular el volumen de fluido necesario para llenar el depósito cilíndrico vacío, necesitamos calcular el volumen de un cilindro, cuya fórmula es:

$$V = \pi \cdot r^2 \cdot h$$

Donde: - r es el radio (en este caso, el radio interno = 3 dm) - h es la altura del cilindro

En el problema se nos proporciona: - Diámetro externo = 10,4 dm - Radio interno = 3 dm

Sin embargo, no se nos proporciona la altura del cilindro, que es esencial para calcular el volumen. Por lo tanto, a Mateo le falta conocer la altura del cilindro para poder calcular la cantidad de fluido necesaria.

Si tuviéramos la altura, el volumen se calcularía así: $V = \pi \cdot (3)^2 \cdot h = 28,27 \cdot h \text{ dm}^3$

Answerlist

- Falso
- Falso
- Verdadero
- Falso

Meta-information

exname: volumen_cilindro_hueco extype: schoice exsolution: 0010 exshuffle: TRUE exsection: Geometría|Volumen|Cilindro